October 2026. Overview and technical summary of the Argus code in this repository (projects/argus, including the engine it is built on). Details: Argus — Documentation, the project README and ENGINE.md.

1. Overview

In one line

Argus lets you ask your cameras what happened, in plain language, and answers with the frames as evidence.

The problem

Sites record far more video than anyone watches. A recorder lets you jump to a time, not to a moment, so finding something means scrubbing through hours of footage. Classic camera AI answers "is X happening right now?" only for rules set up in advance; today's question was usually not one of them.

What Argus does

Step	What happens
Record	Every camera's stream is analysed live: all 80 everyday object types are detected and tracked, each tracked object and each scene is stored as a searchable description, and each person's body pose becomes facts (posture, hands up, facing, colour of top and trousers).
Ask	A question such as *"Was there a white van at the gate yesterday evening?"* is split into *what*, *when* and *which camera*. Time can be plain words ("last friday", "this morning", "6pm to 9pm").
Answer	A language model looks things up in the site's memory, answers in sentences and lists the thumbnails it checked as evidence.
Alert	The same recorder fires alerts on areas and lines drawn on each camera (enter, leave, stay too long, cross), plus *person down* and *hands up* from body poses. Alerts go to webhook or email, filtered by rule, object, hours and cooldown.

Argus says which kind of answer it gives:

- **Counted (exact):** from the detector and tracker. *"How many cars came in today?"*
- **Described (ranked):** visual search by appearance. *"A man in a red jacket near a car?"*
- **Measured (pose):** facts per person. *"Did anyone fall in the warehouse last night?"*
- **Face (on approval only):** *"Did the person in this photo come to the office between 1 and 4?"*

Why it is different

- **Private by design.** Runs on the site's own server; footage never leaves the building. The default chat model is local (Qwen3 4B via Ollama). A cloud model (Claude) is opt-in and only ever receives text (times, cameras, labels), never images or video.
- **No GPU needed.** Detection, search and pose run on CPU through ONNX Runtime.
- **Commercially clean.** Every model ships under a permissive licence (Apache-2.0 or MIT) for code, weights and training data. AGPL/GPL and research-only models are excluded.
- **Works with existing cameras.** Any IP camera with RTSP (Hikvision, Dahua, Reolink, Uniview, Axis presets, or a plain URL).
- **Biometrics off by default.** Face search needs the client's written approval and can be limited to some cameras; face prints are deleted after 7 days.

Status

Working now	Next
Recording and indexing per camera (80 object types + scene snapshots)	Fire and smoke detection (own model, open datasets)
Plain-language search without an LLM (`ask.py`)	A fixed set of test questions to compare chat models
Chat with switchable local or cloud models	Scene captions from a small vision-language model
Main-stream recording and full-resolution crops	Multi-camera timeline (one visit across cameras)
Area / line alerts to webhook and email; chat answers about past alerts	Telegram, WhatsApp, SMS and on-site siren channels
Body poses: posture, hands up, colours; person-down and hands-up alerts	Person-down validated on staged falls at a pilot site
Face search by photo (opt-in)	Chat in the browser; API login

Go-to-market: a pilot on the client's own recorded footage (we index a sample, they ask their own questions, then decide). A partner programme (Silver / Gold / Platinum, 10–30%) is drafted for introducers and CCTV installers.

2. Technical summary

Architecture

```
IP cameras (RTSP)
  ├─ sub stream ──► detect (YOLOX) ─► track (IoU tracker) ─► zone / line rules ─► alerts ─► webhook, email
  │                                          │
  │                                          ├─► CLIP index: objects (every 2 s), scenes + 2x2 tiles (every 5 s)
  │                                          ├─► pose index: 17 keypoints → posture, arms, facing, colours (~1/s)
  │                                          └─► face index: YuNet + SFace, best 5 faces per visit (opt-in)
  └─ main stream ─► ffmpeg: 1-minute MP4s (copy, no re-encode) + keyframes for full-resolution crops
                                          │
                              one SQLite store per site (events, sightings, faces, poses + media)
                                          │
                                          ├─► ask.py     plain-language search, rules only (no LLM)
                                          ├─► chat.py    LLM + tools, answers with evidence
                                          └─► server.py  FastAPI: search, people, events, media, recordings
```

One recorder process per camera; each heavy step (CLIP, pose, faces) runs on its own thread and **skips samples rather than slowing detection**, so a busy scene is sampled more sparsely instead of lagging behind.

Code layout

The code is one Python package, `argus`, in `projects/argus`, with one `uv` environment (Python ≥ 3.11):

- **argus.engine** (~25 modules): detector, tracker, zones, alerts, store, indexer, CLIP / face / pose models, query parser, search, FastAPI server, camera discovery, main-stream recording, Gradio test UI.
- **argus** (the product layer on top of the engine): `record.py`, `ask.py`, `chat.py` (chat CLI), `tools.py` (what the LLM may call), `llm.py` (model backends) and `ui.py` (test UI: record, find a person by photo, chat, reports, and the engine's test bench).

About 8,000 lines of Python and ~125 unit tests (`uv run pytest`). Other folders: `docs/` (documentation, partner programme, presentation), `Brochures/`, `Images/argus/` (brand kit).

Models

Role	Model	Licence	Cost on a laptop CPU
Object detection	YOLOX-Nano (COCO, 80 classes), larger YOLOX variants selectable	Apache-2.0	4 MB model, real time
Visual search	CLIP ViT-B/32 (OpenCLIP, LAION-400M), float32	MIT	~55 ms per crop, 2 KB per vector
Body pose	ViTPose-B, COCO only	Apache-2.0	100–175 ms per person
Faces (opt-in)	YuNet detector + SFace recogniser	MIT / Apache-2.0	128-number face print
Chat (default)	Qwen3 4B via Ollama; others selectable	Apache-2.0	CPU

All vision models run as ONNX graphs with numpy/OpenCV pre- and post-processing: no PyTorch at run time and a straightforward port to C++. PyTorch is only used in one-off export scripts.

How a question is answered

1. Parse. `query.py` turns the sentence into *what / when / camera* with rules (today, yesterday, last friday, after 10pm, last 2 hours, camera names and aliases). No LLM needed.

2. Look up. The chat model gets up to five tools over the store (`people` and `face` only where pose and faces are enabled) and may run up to six rounds of lookups:

Tool	Answers	Kind
`objects`	what was tracked, when, how many; "together with" (same camera within 15 s)	exact
`search`	CLIP text-to-image search over objects and scene tiles	ranked
`people`	posture time, hands up, facing, clothing colours	exact
`alerts`	what the alert rules fired	exact
`face`	visits matching an attached photo (likely / possible)	scored

3. Group and cite. Sightings of the same track are merged into one *visit*; the answer lists the thumbnails it relied on.

Models are config entries, not code: any OpenAI-compatible server (Ollama, llama.cpp, LM Studio, vLLM) or the Anthropic API. Each camera has an `about:` block (location, aliases, what it covers, face quality, normal hours) that goes into the model's instructions.

Storage and API

- **SQLite, one file per site:** `events` (with JPEG snapshots), `sightings` (CLIP vectors), `faces`, `poses`. Pose rows are written only on change plus a 10 s heartbeat (~250 bytes each).
- **Retention:** sightings 30 days, face prints 7 days, recordings 7 days (or sooner when disk is low). Purging runs inside the API server.
- **Search** is brute force with numpy, fine for a pilot; sqlite-vec or pgvector for large sites.
- **API (FastAPI):** `/search`, `/search/face`, `/people`, `/events`, `/media`, `/recordings` (links open the video 3 s before the moment), `/cameras`, `/health`, `/maintenance/purge`.

Deployment footprint

- A standard server, no GPU. Cameras declared in YAML; passwords in `.env`.
- Recording at 1080p, 4 Mbit/s is about **43 GB per camera per day** (needs ffmpeg ≥ 5.1).
- Main-stream keyframes (1 per second) give sharp crops for search and faces at the cost of one extra detection per second per camera.

Known limits

- "Everything" is sampled (objects every 2 s, scenes every 5 s, nothing under 40 px tall); all tunable per site.
- Visual search scores are relative: results are ranked guesses, checked through the evidence.
- A long occlusion can split one visit into two.
- Poses need people ≥ 80 px tall; colours are lost under infrared; sitting and crouching can be confused. Person-down is unit-tested but **not yet validated on real falls**.
- Face thresholds were set on clear photos, not CCTV: calibrate per pilot site.
- Alert rules only cover the 80 detector classes plus the two pose events. New objects (fire, smoke, customer-specific items) need a trained model; rules and channels then apply unchanged.
- The API has **no authentication yet**: keep it on `127.0.0.1` or behind a proxy with login.
- Small local models follow tool instructions less reliably than large ones.